

Prompt Testing Methodology for Notebook Audit

Multi-LLM Comparison Framework and Experiment Design

July 10, 2026

Describes the method for testing audit prompts against heterogeneous LLMs, executing structured notebook audits, and comparing model outputs via a common JSON schema.

1 Introduction

This document presents the methodology for testing and comparing Large Language Models (LLMs) as audit tools for Jupyter notebooks in machine learning and data science contexts. The approach rests on three pillars:

1. A **desktop application** (test-prompts-app) that provides hardware detection, GGUF model download, and a local OpenAI-compatible inference server — enabling both API-based and locally-hosted LLMs to be tested under the same conditions.
2. A **structured audit framework** (six-pass review protocol) that defines what to look for in a notebook, organized into three escalating levels: Conceptual, Methodological, and Implementation.
3. A **cross-LLM comparison method** that treats each model as an interchangeable “sensor” receiving the same fixed prompt and returning the same fixed JSON schema, making outputs directly comparable field-by-field.

Together, these three pillars allow researchers to measure not which model is “correct”, but where models agree, where they diverge, and what the variability patterns reveal about prompt robustness and model-specific blind spots.

2 Current Implementation: test-prompts-app

The **test-prompts-app** is a cross-platform desktop application built with Python 3.14, Flet, llama-cpp-python, and HuggingFace Hub. It serves as the execution platform for the prompt-testing methodology.

2.1 Core Capabilities

Hardware Detection Detects OS, RAM, and GPU information (Metal on macOS, CUDA/ROCm on Linux, CUDA on Windows). Recommends a model parameter size appropriate to available resources.

Model Discovery and Download Queries the HuggingFace Hub for GGUF models compatible with the detected hardware, lists available quantizations with file sizes, and downloads a single .gguf file to a local models directory.

Local Inference Server Starts an in-process LLM server via `llama-cpp-python[server]` and `uvicorn`. Exposes an OpenAI-compatible `/v1/chat/completions` endpoint at a configurable host and port, with GPU layer offload and context size controls.

Multi-OS GPU Support Supports Metal (macOS), CUDA (Linux/Windows), and ROCm (Linux) backends, compiled with platform-specific `CMAKE_ARGS` during installation.

2.2 User Configuration

Settings are persisted as JSON at `~/.test-prompts/config.json`, covering host, port, GPU layers, context size, and the local models directory.

2.3 Experiment Templates

The existing `templates/` directory defines two JSON schemas:

`input-experiment.json` Specifies a prompt to run and a list of target models (each tagged as `API` or `Local`), with an ISO-8601 timestamp.

`output-experiment.json` Captures per-model responses, each with a `delayResponse` timestamp enabling latency computation against the input timestamp.

These templates form the foundation for running controlled, reproducible experiments across multiple LLMs.

3 Notebook Audit Framework

The audit framework is a **backward-looking diagnostic**: it guides a reviewer — human or AI — in systematically assessing the quality, correctness, and risk profile of an existing notebook. It is formalized as a six-pass review protocol, applied sequentially. Each pass has a defined scope and deliverable.

Where a deliverable is a score, the reviewer rates on a three-level scale: **Low Risk**, **Moderate Risk**, **High Risk** — based on the number and severity of unresolved items, not a numeric scale.

3.1 Three-Level Audit Hierarchy

For LLM-based auditing, the six passes are grouped into three levels that escalate in depth:

Level 1 — Conceptual (Passes 1–2): Establishes whether the notebook is structurally coherent and re-runnable before any deep inspection is worth doing.

Level 2 — Methodological (Passes 3–4): Addresses whether the data pipeline and modeling logic are scientifically sound.

Level 3 — Implementation (Passes 5–6): Addresses how the working logic is written and packaged — the most granular, code-level layer.

Each level has dedicated prompt templates (see Section 4) that scope the model’s attention to that level’s specific concerns, preventing scope creep and improving output consistency.

4 Prompt Engineering Evolution

The prompt design for notebook auditing evolved through three iterations (called “Laps”), each increasing in specificity, output-structure demands, and the number of distinct audit checks.

Table 1: The six-pass review protocol.

Pass	Scope	Deliverable
1 — Structural Overview	Cell execution order, section headers, red-flag scan (missing outputs, broken imports, orphaned cells).	Section map, red-flag list, focus-area scope.
2 — Reproducibility	Dependency pinning, random seeds, hard-coded paths, end-to-end re-execution.	Reproducibility risk score (Low/Moderate/High).
3 — Data Integrity	Train/test split ordering, pipeline-level leakage, missing-data handling, ingestion-time validation.	Data pipeline integrity report.
4 — ML Correctness	Evaluation metric appropriateness, cross-validation integrity, hyperparameter tuning boundaries, baseline comparison.	ML correctness checklist.
5 — Code Quality	Repetitive code (>3 similar blocks), dead code, naming quality, bloated cell outputs.	Code quality and smell report.
6 — Deployment Readiness	Artifact export with versioning, inference/training separability, resource documentation, environment completeness, PII concerns.	Deployment readiness score (Low/Moderate/High).

4.1 Lap 1 — Coarse Evaluation

A coarse, two-phase prompt that asks the model to produce documentation (overall purpose, block descriptions, pseudocode, pipeline summary) and then a critical audit (cross-block consistency, redundancy, contradictory logic, missing error handling). The output is free-form text with minimal structure.

4.2 Lap 2 — Refined Evaluation

Introduces explicit formatting requirements: numbered blocks consolidated by type (Environment Setup, then one per computational cell), mandatory metadata per block (Logic Flow, Data Transformations, Model/Algorithm Operations, Key Variables, Side Effects), and a cross-block code quality audit with bullet categories (Coherence, Redundancy, Contradiction). Still free-form, but machine-parseable.

4.3 Lap 3 — Structured Evaluation (Three-Level Prompts)

The current state: each of the three audit levels (Conceptual, Methodological, Implementation) has its own dedicated prompt template. These templates are self-contained, with explicit step-by-step instructions, output-format constraints, and explicit *scope boundaries* that forbid the model from drifting into other audit levels’ territory.

This is the prompt form used in the cross-LLM comparison methodology.

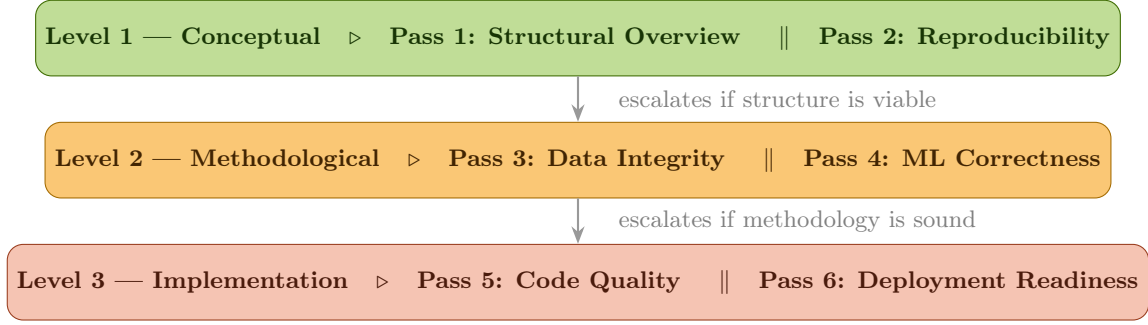


Figure 1: Three-level audit hierarchy. Each level gates the next: Level 1 must confirm structural coherence before Level 2 inspects methodology; Level 2 must confirm methodological soundness before Level 3 inspects code quality.

5 Sensor Prompt Methodology

The final iteration of the prompt strategy treats each LLM as an **interchangeable sensor**. A single, fixed prompt — the “sensor prompt” — is sent to every model under test, along with the same notebook content. The prompt contains strict constraints:

- No scoring or severity labels (the sensor only extracts evidence).
- No aggregating multiple issues into one observation.
- Return only JSON matching the **SensorOutput** schema.
- Refuse a monolithic audit (return a **chunking-required** flag) if the notebook exceeds context limits.

5.1 SensorOutput JSON Schema

The single output shape every model must produce:

overall_purpose (string) 2–3 sentence summary of the notebook’s goal.

pipeline_summary (string) 3–5 sentence summary of block dependencies and data flow.

observations (array) One entry per (criterion, cell) pair. Each entry contains:

- **criterion_id** — e.g., "1.4", "2.1"
- **cell_id** — stable cell identifier
- **applicability** — **IN_SCOPE** or **NOT-APPLICABLE**
- **observation** — factual statement, no judgement words
- **code_snippet** — exact snippet supporting the claim
- **line_range** — e.g., "C005:10-15"

complex_functions (array) Non-trivial functions translated to pseudocode. Each entry: **cell_id**, **function_name**, **pseudocode**.

cross_reference_discrepancies (array) Mismatches between the documented pipeline summary and the actual code.

5.2 Audit Criteria

The sensor prompt scans two fixed criteria groups:

Table 2: Audit criteria applied by the sensor prompt.

ID	Criterion	What it checks
1.1	Workflow Order	Whether cells follow a logical top-to-bottom progression matching the section structure.
1.2	Execution Order	Strictly linear execution; flag cells that depend on later cells.
1.3	Code Readability	Naming clarity, logical grouping, absence of monolithic cells.
1.4	Documentation	Markdown cells explaining intent and expected output for every code cell.
2.1	Environment Variables	Hardcoded paths, credentials, or environment-specific assumptions.
2.2	Reliable Data Handling	Data validation at ingestion, consistent split handling, leakage avoidance.
2.3	Atomic/Reusable Cells	Cell independence, explicit variable passing, no hidden state.
2.4	Reproducibility Controls	Random seeds set globally and consistently before any stochastic operation.
2.5	Caching/Fast Reruns	Mechanisms to avoid redundant computation on re-execution.

6 LLMs Under Test

6.1 Model Classification

Models are classified into two types in the experiment configuration:

API Remote, cloud-hosted models accessed through a provider’s HTTP endpoint (e.g., GPT-4, Claude). The app sends the prompt via a provider abstraction layer.

Local Self-hosted models running on the user’s hardware through the in-process llama.cpp server. The same OpenAI-compatible `/v1/chat/completions` interface is used regardless of whether the model is local or remote.

6.2 Experiment Configuration

The extended `input-experiment.json` template ties together the prompt, the notebook, and the list of target models:

```
{
  "timestamp": "2026-07-10T20:00:00Z",
  "prompt": "You are a factual evidence-extraction sensor...",
  "notebookPath": "notebooks/example.ipynb",
  "models": [
    { "name": "gpt-4", "type": "API" },
  ],
}
```

```

{ "name": "claude-sonnet-5", "type": "API" },
{ "name": "llama-3-8b", "type": "Local", "quantization": "q4_0" }
]
}

```

6.3 Output Collection

The corresponding `output-experiment.json` captures each model's response as a parsed `SensorOutput` object, together with timing data:

```

{
  "timestamp": "2026-07-10T20:00:00Z",
  "outputs": [
    {
      "model": "gpt-4",
      "type": "API",
      "response": { "overall_purpose": "...", "observations": [...] },
      "delayResponse": "2026-07-10T20:00:04Z"
    }
  ]
}

```

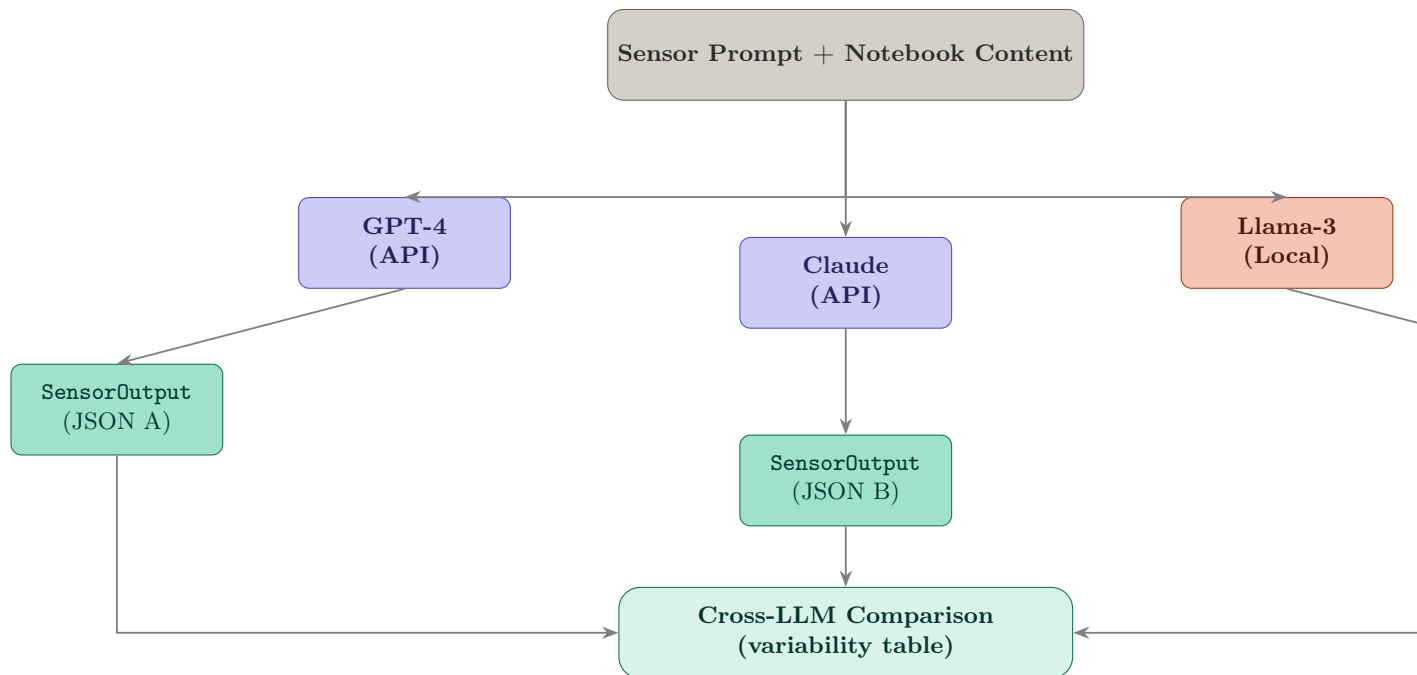


Figure 2: Execution flow: the same structured prompt and notebook are dispatched to each LLM; each returns a `SensorOutput` JSON object; all outputs are then compared in a unified variability table.

7 Cross-LLM Comparison

Because every model returns the same `SensorOutput` schema, outputs from N models produce N structurally identical JSON files that can be compared field-by-field. This is the core of the variability analysis.

7.1 Comparison Table

The comparison pivots the **observations** arrays from all models into one table, keyed by **criterion_id** + **cell_id**, with one column per LLM:

Table 3: Example comparison table. A blank cell means the model did not emit an observation for that criterion–cell pair — itself a meaningful signal.

Criterion	Cell	GPT-4	Claude	Llama-3	Variability
2.4	C008	Seeds set for random, numpy, torch	Seeds set for random, numpy, torch	(<i>absent</i>)	partial (2/3)
2.1	C005	DATA_URL and RNG_SEED hardcoded	Only RNG_SEED flagged; DATA_URL missed	Config values hardcoded	divergent
1.1	—	NOT-APPLICABLE	IN_SCOPE	IN_SCOPE	applicability disagreement
1.3	C003	Clear, well-named variables	Clear, well-named variables	Clear, well-named variables	consistent

7.2 Variability Types

Each comparison row is classified by the nature of disagreement:

consistent All models agree in both coverage and content.

missing-in-one One or more models skipped a cell/criterion the others covered.

applicability-disagreement Models disagree on whether a criterion is **IN_SCOPE** vs **NOT-APPLICABLE** for a cell.

text-divergence All models reported something, but the factual content differs materially (different snippet, different scope).

7.3 Variability Score

A simple scalar metric per row and per notebook:

$$\text{variabilityScore}_{\text{row}} = \frac{\text{number of models disagreeing with the majority}}{N_{\text{models}}}$$

Aggregated across all rows, this gives a single “how much do these models agree” number per notebook, useful for deciding whether a single model’s audit alone is trustworthy enough to skip cross-checking.

7.4 comparison-experiment.json Artifact

This structured artifact stores the full comparison for reproducibility:

```
{
  "timestamp": "2026-07-10T20:05:00Z",
```

```

"notebookPath": "notebooks/example.ipynb",
"models": ["gpt-4", "claude-sonnet-5", "llama-3-8b"],
"comparisonRows": [
  {
    "criterion_id": "2.4",
    "cell_id": "C008",
    "byModel": {
      "gpt-4": { "applicability": "IN_SCOPE", "observation": "..."},
      "claude": { "applicability": "IN_SCOPE", "observation": "..."},
      "llama-3": null
    },
    "variabilityScore": 0.33,
    "variabilityType": "missing-in-one"
  }
],
"crossReferenceDiscrepancyDiff": {
  "onlyIn": {
    "gpt-4": [],
    "claude": ["pipeline_summary claims caching; C024 shows none"],
    "llama-3": []
  }
}
}

```

8 Method Summary

The complete methodology for testing prompts and comparing LLMs as notebook auditors consists of the following pipeline:

1. **Set up the execution environment.** Use the test-prompts-app to detect hardware, download GGUF models for local inference, and configure API credentials for remote models.
2. **Define the experiment.** Populate `input-experiment.json` with the sensor prompt (or a level-specific audit prompt), the path to the target notebook, and the list of models under test.
3. **Dispatch to all models.** The app sends the same prompt + notebook to every model. API models are called in parallel; local models share the server slot and are dispatched sequentially.
4. **Collect structured outputs.** Each response is parsed and validated against the `SensorOutput` schema. Raw responses are preserved for traceability.
5. **Compute the comparison.** Pivot the `observations` arrays into a unified table keyed by `criterion_id + cell_id`, with one column per model. Classify each row by variability type and compute a variability score.
6. **Analyse variability.** Review rows flagged as `divergent`, `missing-in-one`, or `applicability-disagreement` — these are where a manual reviewer’s attention is most valuable. Consistent rows may be accepted with lower scrutiny.
7. **Store the comparison artifact.** Write `comparison-experiment.json` to disk for reproducibility and to enable comparison across notebooks and prompt versions.

9 Project Structure (Relevant Components)


```

test/prompts/
  app/
    api/
      local.py      -- hardware detection, HF browser, GGUF download,
                     AsyncLlamaServer (in-process inference server)
      utils.py      -- provider abstraction (dispatches prompt to
                     both API and Local models)
      compare.py    -- builds comparison rows / variability metrics
                     from a set of SensorOutput JSON files
    UI/
      app.py        -- Hardware / Models / Server / Settings / Compare
                     tabs
  templates/
    input-experiment.json      -- prompt + notebook + model list
    output-experiment.json     -- per-model SensorOutput responses
    comparison-experiment.json -- per-criterion/cell diff across models
  context/
    initial-idea.md           -- original multi-LLM comparison concept
    better-idea.md            -- refined sensor-prompt and cross-LLM design
    proposal/
      prompt-testing-methodology.tex -- this document
  docs/                       -- Sphinx autodoc for the app codebase

```

End of document.